Assignment - 7

```
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Conv2D, MaxPooling2D,
                                    Flatten, Dense, Dropout
import numpy as np
import matplotlib.pyplot as plt
import cv2
from tensorflow.keras import backend as k

(x_train, y_train), (x_test, y_test) = cifar10.load_data()
x_train = x_train.astype('float32')/255.0
x_test = x_test.astype('float32')/255.0
y_train = tf.keras.utils.to_categorical(y_train,10)
y_test = tf.keras.utils.to_categorical(y_test,10)
def build_cifar_model():
    model = Sequential([
        Conv2D(32,(3,3), activation='relu', padding='same',
               input_shape=(32,32,3)),
```

Assignment - 7

```
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Conv2D, MaxPooling2D,
                                    Flatten, Dense, Dropout
import numpy as np
import matplotlib.pyplot as plt
import cv2
from tensorflow.keras import backend as k

(x_train, y_train), (x_test, y_test) = cifar10.load_data()
x_train = x_train.astype('float32')/255.0
x_test = x_test.astype('float32')/255.0
y_train = tf.keras.utils.to_categorical(y_train,10)
y_test = tf.keras.utils.to_categorical(y_test,10)
def build_cifar_model():
    model = Sequential([
        Conv2D(32,(3,3), activation='relu', padding='same',
               input_shape=(32,32,3)),
```

```
    Conv2D(32,(3,3), activation='relu'),
    MaxPooling2D((2,2)),
    Dropout(0.25),

    Conv2D(64,(3,3), activation='relu', padding='same'),
    Conv2D(64,(3,3), activation='relu'),
    MaxPooling2D((2,2)),
    Dropout(0.25),

    Flatten(),
    Dense(512, activation='relu'),
    Dropout(0.5),
    Dense(10, activation='Softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy',
              metrics=['accuracy'])

return model

cifar_model = build_cifar_model()
cifar_model.fit(x_train, y_train, epochs=15, batch_size=64,
                validation_data=(x_test, y_test), verbose=1)
```

```
def make_gradcam_heatmap(img_array, model, last_conv_layer_
        name, pred_index = None):

    input_shape = model.input_shape[1:]
    inputs = tf.keras.Input(shape=input_shape)

    x = inputs
    target_layer_output = None
    for layer in model.layers:
        x = layer(x)
        if layer.name == last_conv_layer_name:
            target_layer_output = x

    grad_model = tf.keras.Model(inputs, [target_layer_output, x])
    with tf.GradientTape() as tape:
        last_conv_layer_output, preds = grad_model(img_array)
        if pred_index is None:
            pred_index = tf.argmax(preds[0])
        class_channel = preds[:, pred_index]
    grads = tape.gradient(class_channel, last_conv_layer_output)
    pooled_grads = tf.reduce_mean(grads, axis=(0,1,2))
    last_conv_layer_output = last_conv_layer_output[0]
```

```
heatmap = last_conv_layer_output @ pooled_grads[..., tf.newaxis]
heatmap = tf.squeeze(heatmap)

heatmap = np.maximum(heatmap, 0) / np.max(heatmap) + 1e-10)
return heatmap.numpy() if hasattr(heatmap, "numpy")
                       else heatmap

def show_explainability(model, img, true_label, last_conv_layer_name)
    img_array = np.expand_dims(img, axis=0)
    preds = model.predict(img_array, verbose=0)
    pred_class = np.argmax(preds[0])
    plt.figure(figsize=(15,5))
    plt.subplot(1,3,1)
    plt.imshow(img)
    plt.title(f"Original\nTrue: {true_label}\nPred:
                          {pred_class}")
    plt.axis('off')

heatmap = make_gradcam_heatmap(img_array, model,
  last_conv_layer_name, pred_index=pred_class)
heatmap_resized = cv2.resize(heatmap, (img.shape[1], img.shape[0]))
plt.subplot(1,3,2)
plt.imshow(heatmap_resized, cmap='jet')
```

```
plt.title("Grad-CAM Heatmap")
plt.axis('off')

heatmap_colored = cv2.applyColorMap(np.uint8(255*
    heatmap_resized), cv2.COLORMAP_JET)
img_255 = np.uint8(255 * img) if img.max() <= 1.0
                              else np.uint8(img)
superimposed = cv2.addWeighted(img_255, 0.6, heatmap_
    colored, 0.4, 0)

plt.subplot(1,3,3)
plt.imshow(superimposed)
plt.title("Superimposed")
plt.axis('off')
plt.tight_layout()
plt.show()

for i in range(5):
    show_explainability(cifar_model, x_test[i],
        cifar_class_names[np.argmax(y_test[i])],
        last_conv_layer_name=last_conv_layer)
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

train_datagen = ImageDataGenerator(rescale=1./255, validation_split=0.2)

train_generator = train_datagen.flow_from_directory(
    '/home/raidul/Desktop/data/faces',
    target_size=(64,64),
    batch_size=32,
    class_mode='categorical',
    subset='training')

val_generator = train_datagen.flow_from_directory(
    '/home/raidul/Desktop/data/faces',
    target_size=(64,64),
    batch_size=32,
    class_mode='categorical',
    subset='validation')

face_model = Sequential([
    Conv2D(32,(3,3), activation='relu', input_shape=(64,64,3)),
    MaxPooling2D(2,2),
    Conv2D(64,(3,3), activation='relu'),
    MaxPooling2D(2,2),
    Conv2D(128,(3,3), activation='relu'),
```

```
MaxPooling2D(2,2)
Flatten(),
Dense(256, activation='relu')
Dropout(0.5),
Dense(train_generator.num_classes, activation='softmax')
])

face_model.compile(optimizer='adam', loss='categorical_
    crossentropy', metrics=['accuracy'])
face_model.fit(train_generator, epochs=10, validation_data
    =val_generator)

test_img_path='/home/saidul/Desktop/data/faces/Saidul_0.jpg'
test_img=tf.keras.preprocessing.image.load_img(test_img_
    path, target_size=(64,64))
test_array=tf.keras.preprocessing.image.img_to_array(test_img)

show_explainability(face_model, test_array, f"Person {np.argmax(
    face_model.predict(np.expand_dims(test_array,0), verbose=0)
    last_conv_layer_name='conv2d_8')
```

(1)

$$I = \begin{bmatrix} 1 & 2 & 3 & 2 & 1 \\ 0 & 1 & 2 & 1 & 0 \\ 1 & 3 & 1 & 3 & 1 \\ 0 & 1 & 2 & 1 & 0 \\ 1 & 2 & 3 & 2 & 1 \end{bmatrix}$$

$$f_1 = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad f_2 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad f_3 = \begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix}$$

weights $w_c = [2, -1, 1]$

a. Global Average pooling

$F_1 = 2.5, \quad F_2 = 0.5, \quad F_3 = 1.0$

b. Class Score

$$S_c = 2(2.5) + (-1)(0.5) + 1(1.0) = 5.5$$

c. Class Activation Map (CAM)

$$M_c = \begin{bmatrix} 4 & 4 \\ 5 & 7 \end{bmatrix}$$

d. Normalized CAM

$$M' = \begin{bmatrix} 0 & 0 \\ 0.2 & 1 \end{bmatrix}$$

e. The 2×2 CAM is upsampled (using bilinear interpolation or nearest neighbor) to the original 5×5 resolution. Each value in the CAM corresponds to a larger receptive field/region in the input image. The highest activated

highlights the central bottom region of the original image.

(2) Set 1: $w^c = [1, 2, 3]$

CAM: $\begin{bmatrix} 4 & 9 \\ 11 & 16 \end{bmatrix} \rightarrow \begin{bmatrix} 0 & 0.917 \\ 0.583 & 1 \end{bmatrix}$

Set 2: $w^c = [3, -2, 1]$

CAM: $\begin{bmatrix} 13 & 1 \\ 7 & 14 \end{bmatrix} \rightarrow \begin{bmatrix} 0.923 & 0 \\ 0.462 & 1 \end{bmatrix}$

Set 3: $w^c = [-1, 3, 2]$

CAM: $\begin{bmatrix} 7 & -1 \\ 14 & 11 \end{bmatrix} \rightarrow \begin{bmatrix} 0.533 & 0 \\ 1 & 0.8 \end{bmatrix}$

(3) Using the original feature maps and computed gradients:

Importance weight (α^c): $\alpha_1 = 1.5$, $\alpha_2 = -0.5$

$\alpha_3 = 2.0$

Grad-CAM before ReLU: $\begin{bmatrix} 5.5 & 4.5 \\ 4 & 8 \end{bmatrix}$

After ReLU: Same (no negative values)

The model focuses strongly on the bottom-right regi

4. Grad-CAM when multiple FC layers exist:

Grad-CAM works seamlessly even with multiple fully connected layers. We compute the gradient of the target class score with respect to the last convolutional layers feature maps. These gradients naturally flow back through all FC layers. Then we apply Global Average pooling on the gradients to get α_k^c and proceed as usual.

Advantage: Grad-CAM is architecture agnostic (works with complex heads), unlike CAM.

5. Visualize Before vs After ReLU on Heatmaps.

Before: contains both positive and negative values.
→ noisy heatmap (red + blue regions)

After: Only positive values remain → clean heatmap showing only features that increase the class score

4. Grad-CAM when multiple FC layers exist:

Grad-CAM works seamlessly even with multiple fully connected layers. We compute the gradient of the target class score with respect to the last convolutional layers feature maps. These gradients naturally flow back through all FC layers. Then we apply Global Average pooling on the gradients to get α_k^c and proceed as usual.

Advantage: Grad-CAM is architecture agnostic (works with complex heads), unlike CAM.

5. Visualize Before vs After ReLU on Heatmaps.

Before: contains both positive and negative values → noisy heatmap (red + blue regions)

After: only positive values remain → clean heatmap showing only features that increase the class score

Conclusion: ReLU improves interpretability by removing suppressing features.

6. Without ReLU, we get full attribution:

positive values = features that support the prediction

Negative values = features that oppose the prediction

Thus gives complete information but produces noisier visualizations. Standard Grad-CAM uses ReLU for cleaner, class-specific explanation

7. IG computation:

$F(x_1, x_2) = x_1^2 + 2x_2$, $x_2 \in (2,3)$, $x' = (0,0)$, $x_2 = 10$

$IG_1 = 4.9$

$IG_2 = 6.0$

Total attribution = 10.9 (close to actual output difference)

8. 3 custom Sets for IG

Set 1: $F = x_1^2 + x_2$

$x = (3, 4) \rightarrow IG_1 \approx 9.0$

$IG_2 \approx 16.0$

Set 2: $F = 3x_1 + 2x_1x_2$

$x = (2, 3)$

$\rightarrow IG_1 \approx 15.0$, $IG_2 \approx 12.0$

~~$IG \approx 12$~~

Set 3: $F = \sin(x_1) + x_2^2$, $x = (1.57, 2)$

$\rightarrow IG_1 \approx 1.0$, $IG_2 \approx 4.0$

9. Grad-CAM vs IG:

Criteria	Grad-CAM	Integrated Gradients (IG)
Purpose	Visual explanation (where?)	Feature attribution (how much?)
Best for	Image data (spatial heatmaps)	Any differentiable model
Computational cost	low	Higher

Criteria	Grad. CAM	IG
Mathematical Property	Good localization	Axiomatic (complete attribution)
Output	2D Heatmap	Scalar importance per input dimension

In [ ]:
```
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dr
import numpy as np
import matplotlib.pyplot as plt
import cv2

print("TensorFlow version:", tf.__version__)
```

```
TensorFlow version: 2.16.2
```

In [3]:
```
# Load CIFAR-10
(x_train, y_train), (x_test, y_test) = cifar10.load_data()
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)

# Simple but good enough CNN for CIFAR-10
def build_cifar_model():
    model = Sequential([
        Conv2D(32, (3,3), activation='relu', padding='same', input_shape=(32
        Conv2D(32, (3,3), activation='relu'),
        MaxPooling2D((2,2)),
        Dropout(0.25),

        Conv2D(64, (3,3), activation='relu', padding='same'),
        Conv2D(64, (3,3), activation='relu'),
        MaxPooling2D((2,2)),
        Dropout(0.25),

        Flatten(),
        Dense(512, activation='relu'),
        Dropout(0.5),
        Dense(10, activation='softmax')
    ])
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics
    return model

cifar_model = build_cifar_model()
cifar_model.fit(x_train, y_train, epochs=15, batch_size=64, validation_data=
```

```
/home/saidul/Desktop/4-2/Deep Learning/myenv/lib/python3.12/site-packages/ke
ras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `
input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instea
d.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
/home/saidul/Desktop/4-2/Deep Learning/myenv/lib/python3.12/site-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
Epoch 1/15
```

```
/home/saidul/Desktop/4-2/Deep Learning/myenv/lib/python3.12/site-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
Epoch 1/15
```

```
2026-05-09 19:59:46.216259: W external/local_tsl/tsl/framework/cpu_allocator_impl.cc:83] Allocation of 614400000 exceeds 10% of free system memory.
```

```
782/782 ━━━━━━━━━━━━━━━━━━━━ 117s 144ms/step - accuracy: 0.4327 - loss: 1.55
33 - val_accuracy: 0.5826 - val_loss: 1.1826
Epoch 2/15
782/782 ━━━━━━━━━━━━━━━━━━━━ 114s 145ms/step - accuracy: 0.5931 - loss: 1.15
01 - val_accuracy: 0.6468 - val_loss: 1.0049
Epoch 3/15
782/782 ━━━━━━━━━━━━━━━━━━━━ 110s 141ms/step - accuracy: 0.6469 - loss: 0.99
46 - val_accuracy: 0.6923 - val_loss: 0.8702
Epoch 4/15
782/782 ━━━━━━━━━━━━━━━━━━━━ 109s 139ms/step - accuracy: 0.6847 - loss: 0.89
62 - val_accuracy: 0.7153 - val_loss: 0.8068
Epoch 5/15
782/782 ━━━━━━━━━━━━━━━━━━━━ 106s 135ms/step - accuracy: 0.7117 - loss: 0.82
35 - val_accuracy: 0.7280 - val_loss: 0.7812
Epoch 6/15
782/782 ━━━━━━━━━━━━━━━━━━━━ 114s 146ms/step - accuracy: 0.7291 - loss: 0.76
96 - val_accuracy: 0.7385 - val_loss: 0.7564
Epoch 7/15
782/782 ━━━━━━━━━━━━━━━━━━━━ 110s 141ms/step - accuracy: 0.7401 - loss: 0.73
47 - val_accuracy: 0.7458 - val_loss: 0.7261
Epoch 8/15
782/782 ━━━━━━━━━━━━━━━━━━━━ 108s 138ms/step - accuracy: 0.7549 - loss: 0.69
48 - val_accuracy: 0.7521 - val_loss: 0.7030
Epoch 9/15
782/782 ━━━━━━━━━━━━━━━━━━━━ 109s 139ms/step - accuracy: 0.7655 - loss: 0.66
61 - val_accuracy: 0.7660 - val_loss: 0.6750
Epoch 10/15
782/782 ━━━━━━━━━━━━━━━━━━━━ 110s 140ms/step - accuracy: 0.7760 - loss: 0.63
53 - val_accuracy: 0.7704 - val_loss: 0.6695
Epoch 11/15
782/782 ━━━━━━━━━━━━━━━━━━━━ 104s 133ms/step - accuracy: 0.7838 - loss: 0.61
70 - val_accuracy: 0.7580 - val_loss: 0.7198
Epoch 12/15
782/782 ━━━━━━━━━━━━━━━━━━━━ 116s 149ms/step - accuracy: 0.7881 - loss: 0.59
65 - val_accuracy: 0.7761 - val_loss: 0.6585
Epoch 13/15
782/782 ━━━━━━━━━━━━━━━━━━━━ 108s 139ms/step - accuracy: 0.7951 - loss: 0.58
39 - val_accuracy: 0.7576 - val_loss: 0.7089
Epoch 14/15
782/782 ━━━━━━━━━━━━━━━━━━━━ 107s 137ms/step - accuracy: 0.8008 - loss: 0.56
05 - val_accuracy: 0.7867 - val_loss: 0.6181
Epoch 15/15
782/782 ━━━━━━━━━━━━━━━━━━━━ 99s 127ms/step - accuracy: 0.8081 - loss: 0.541
0 - val_accuracy: 0.7761 - val_loss: 0.6556
```

Out[3]: <keras.src.callbacks.history.History at 0x7834a3d492b0>

In [14]:
```python
# Check layer names
for layer in cifar_model.layers:
    if 'conv' in layer.name:
        print(layer.name)
```

```
conv2d
conv2d_1
conv2d_2
conv2d_3
```

In [40]:
```
def make_gradcam_heatmap(img_array, model, last_conv_layer_name, pred_index=
    # 1. Dynamically get the input shape from the model
    # model.input_shape is usually (None, 64, 64, 3)
    input_shape = model.input_shape[1:]
    inputs = tf.keras.Input(shape=input_shape)

    # 2. Re-route through layers (Dynamic rebuilding)
    x = inputs
    target_layer_output = None
    for layer in model.layers:
        x = layer(x)
        if layer.name == last_conv_layer_name:
            target_layer_output = x

    # 3. Create the gradient model
    grad_model = tf.keras.Model(inputs, [target_layer_output, x])

    # 4. GradientTape logic
    with tf.GradientTape() as tape:
        last_conv_layer_output, preds = grad_model(img_array)
        if pred_index is None:
            pred_index = tf.argmax(preds[0])
        class_channel = preds[:, pred_index]

    grads = tape.gradient(class_channel, last_conv_layer_output)
    pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))

    # 5. Generate heatmap
    last_conv_layer_output = last_conv_layer_output[0]
    heatmap = last_conv_layer_output @ pooled_grads[..., tf.newaxis]
    heatmap = tf.squeeze(heatmap)

    # Normalize
    heatmap = np.maximum(heatmap, 0) / (np.max(heatmap) + 1e-10)
    return heatmap.numpy() if hasattr(heatmap, "numpy") else heatmap
```

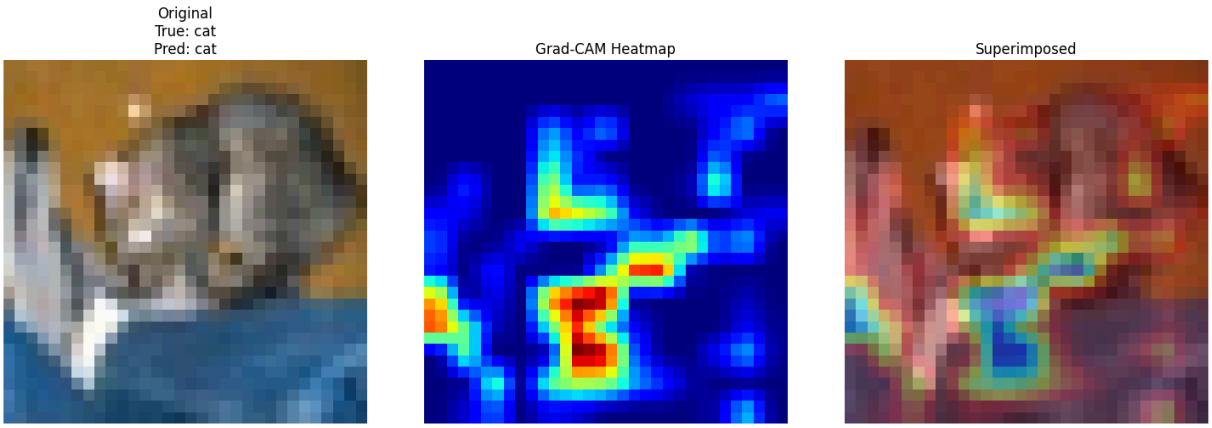
In [45]:
```
cifar_class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
                     'dog', 'frog', 'horse', 'ship', 'truck']

last_conv_layer = "conv2d_3"   # Confirmed from your list
```

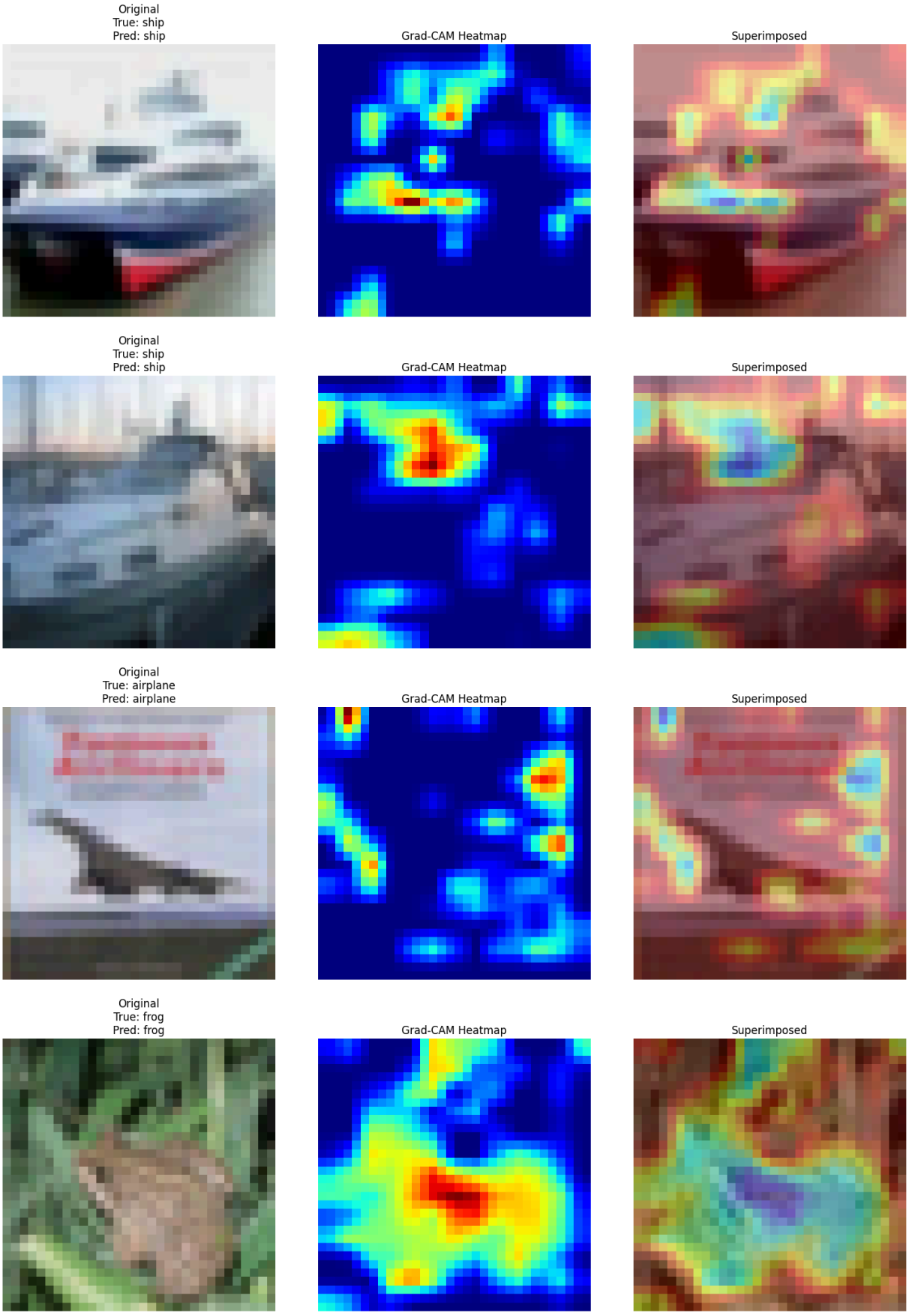
In [46]:
```
def show_explainability(model, img, true_label, last_conv_layer_name):
    img_array = np.expand_dims(img, axis=0)

    # Get prediction
    preds = model.predict(img_array, verbose=0)
    pred_class = np.argmax(preds[0])

    plt.figure(figsize=(15, 5))

    plt.subplot(1, 3, 1)
    plt.imshow(img)
    plt.title(f"Original\nTrue: {true_label}\nPred: {cifar_class_names[pred_
    plt.axis('off')
```

```
    # Generate heatmap
    heatmap = make_gradcam_heatmap(img_array, model, last_conv_layer_name, p
    heatmap_resized = cv2.resize(heatmap, (img.shape[1], img.shape[0]))

    plt.subplot(1, 3, 2)
    plt.imshow(heatmap_resized, cmap='jet')
    plt.title("Grad-CAM Heatmap")
    plt.axis('off')

    # Superimposed
    heatmap_colored = cv2.applyColorMap(np.uint8(255 * heatmap_resized), cv2
    # Convert image to 0-255 uint8 for OpenCV
    img_255 = np.uint8(255 * img) if img.max() <= 1.0 else np.uint8(img)
    superimposed = cv2.addWeighted(img_255, 0.6, heatmap_colored, 0.4, 0)


    plt.subplot(1, 3, 3)
    plt.imshow(superimposed)
    plt.title("Superimposed")
    plt.axis('off')

    plt.tight_layout()
    plt.show()
```

In [47]:
```
print("Generating Grad-CAM explanations for CIFAR-10...")

for i in range(5):
    show_explainability(cifar_model, x_test[i],
                        cifar_class_names[np.argmax(y_test[i])],
                        last_conv_layer_name=last_conv_layer)
```

```
Generating Grad-CAM explanations for CIFAR-10...
```





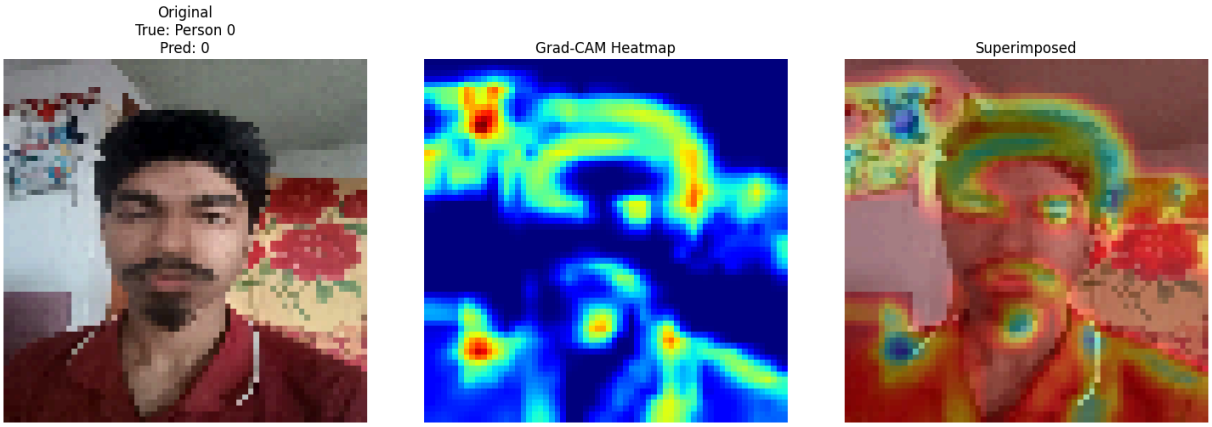
In [43]:
```
# Load your face dataset (same structure as previous assignments)
# Assume you have folder: ../data/faces/ with subfolders person_0, person_1,
```

```python
from tensorflow.keras.preprocessing.image import ImageDataGenerator

train_datagen = ImageDataGenerator(rescale=1./255, validation_split=0.2)

train_generator = train_datagen.flow_from_directory(
    '/home/saidul/Desktop/4-2/Deep Learning/data/faces',
    target_size=(64, 64),
    batch_size=32,
    class_mode='categorical',
    subset='training')

val_generator = train_datagen.flow_from_directory(
    '/home/saidul/Desktop/4-2/Deep Learning/data/faces',
    target_size=(64, 64),
    batch_size=32,
    class_mode='categorical',
    subset='validation')

# Build simple face classifier
face_model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(64,64,3)),
    MaxPooling2D(2,2),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2,2),
    Conv2D(128, (3,3), activation='relu'),
    MaxPooling2D(2,2),
    Flatten(),
    Dense(256, activation='relu'),
    Dropout(0.5),
    Dense(train_generator.num_classes, activation='softmax')
])

face_model.compile(optimizer='adam', loss='categorical_crossentropy', metric
face_model.fit(train_generator, epochs=10, validation_data=val_generator)
```

```
Found 28 images belonging to 7 classes.
Found 7 images belonging to 7 classes.
Epoch 1/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step - accuracy: 0.2500 - loss: 1.9576 - val_
accuracy: 0.1429 - val_loss: 1.9196
Epoch 2/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 413ms/step - accuracy: 0.1786 - loss: 1.9064 - v
al_accuracy: 0.1429 - val_loss: 1.8629
Epoch 3/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 293ms/step - accuracy: 0.2143 - loss: 1.8099 - v
al_accuracy: 0.5714 - val_loss: 1.7940
Epoch 4/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 258ms/step - accuracy: 0.3214 - loss: 1.8309 - v
al_accuracy: 0.7143 - val_loss: 1.7104
Epoch 5/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 264ms/step - accuracy: 0.3571 - loss: 1.8076 - v
al_accuracy: 0.8571 - val_loss: 1.6427
Epoch 6/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 284ms/step - accuracy: 0.4286 - loss: 1.6479 - v
al_accuracy: 0.5714 - val_loss: 1.5642
Epoch 7/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 307ms/step - accuracy: 0.4286 - loss: 1.5329 - v
al_accuracy: 0.5714 - val_loss: 1.4512
Epoch 8/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 266ms/step - accuracy: 0.3571 - loss: 1.4616 - v
al_accuracy: 0.7143 - val_loss: 1.3030
Epoch 9/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 276ms/step - accuracy: 0.5357 - loss: 1.3134 - v
al_accuracy: 0.7143 - val_loss: 1.1595
Epoch 10/10
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 274ms/step - accuracy: 0.6429 - loss: 1.2348 - v
al_accuracy: 0.7143 - val_loss: 1.0462
```

Out[43]: <keras.src.callbacks.history.History at 0x78343d425fa0>

In [44]:
```
# Test on one of your face images
test_img_path = '/home/saidul/Desktop/4-2/Deep Learning/data/faces/person_0/
test_img = tf.keras.preprocessing.image.load_img(test_img_path, target_size=
test_array = tf.keras.preprocessing.image.img_to_array(test_img) / 255.0

show_explainability(face_model, test_array,
                    f"Person {np.argmax(face_model.predict(np.expand_dims(tes
                    last_conv_layer_name="conv2d_8")   # Update layer name af
```


Original
True: Person 0
Pred: 0
Grad-CAM Heatmap
Superimposed

Mathematical Problem-1

CSE4261: Neural Network and Deep Learning
Department of Computer Science and Engineering
University of Rajshahi

---

1. Assume a CNN-based 3-class classifier which can be used for CAM.
   Let a 5×5 image be passed through the assumed CNN-based 3-class classifier.

$$I = \begin{bmatrix} 1 & 2 & 3 & 2 & 1 \\ 0 & 1 & 2 & 1 & 0 \\ 1 & 3 & 4 & 3 & 1 \\ 0 & 1 & 2 & 1 & 0 \\ 1 & 2 & 3 & 2 & 1 \end{bmatrix}$$

   Let the last convolutional layer of a CNN has three feature maps (size 2×2):

$$f_1 = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad f_2 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad f_3 = \begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix}$$

   Let the fully connected layer weights (for class c): $w^c = [2, -1, 1]$

   a. Compute Global Average Pooling (GAP) outputs F_1, F_2, F_3.
   b. Compute the class score: $S_c = \sum_k w_k^c F_k$
   c. Compute the Class Activation Map: $M_c(x, y) = \sum_k w_k^c f_k(x, y)$
   d. Normalize the CAM: $M' = \frac{M - \min(M)}{\max(M) - \min(M)}$
   e. Explain how this 2×2 CAM would be mapped back to the 5×5 input image.

2. Show steps of CAM for your own three sets of $\{I, f_1, f_2, f_3, w_i\}$.

3. Show steps of Grad-CAM for the above network for the above image.

4. Show Grad-CAM when multiple FC layers exist.

5. Visualize before vs after ReLU as heatmaps.

6. Show what happens if we DON'T use ReLU (full attribution).

7. Say model $F(x_1, x_2) = x_1^2 + 2x_2$, input $x = (2, 3)$, baseline $x' = (0, 0)$, $m = 10$. Compute the Integrated Gradients (IG) using a discrete approximation. Show all steps as in a real implementation of the IG algorithm.

8. Show all steps as in a real implementation of the IG algorithm for your own three sets of $\{F, x, x', m\}$.

9. Compare Grad-CAM vs Integrated Gradients (IG) using the same situation.

CSE4261: Neural Network and Deep Learning

1. Solution for the Given Example

Given Input Image (5×5):

$$I = \begin{bmatrix} 1 & 2 & 3 & 2 & 1 \\ 0 & 1 & 2 & 1 & 0 \\ 1 & 3 & 4 & 3 & 1 \\ 0 & 1 & 2 & 1 & 0 \\ 1 & 2 & 3 & 2 & 1 \end{bmatrix}$$

Feature Maps (2×2):

$$f_1 = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix},\ f_2 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix},\ f_3 = \begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix}$$

Weights: $w^c = [2, -1, 1]$

a. Global Average Pooling (GAP)
$F_1 = 2.5$, $F_2 = 0.5$, $F_3 = 1.0$

b. Class Score
$S_c = 2(2.5) + (-1)(0.5) + 1(1.0) = \mathbf{5.5}$

c. Class Activation Map (CAM)

$$M_c = \begin{bmatrix} 4 & 4 \\ 5 & 9 \end{bmatrix}$$

d. Normalized CAM

$$M' = \begin{bmatrix} 0 & 0 \\ 0.2 & 1 \end{bmatrix}$$

e. Mapping 2×2 CAM back to 5×5 Image
The 2×2 CAM is upsampled (using bilinear interpolation or nearest neighbor) to the original 5×5 resolution. Each value in the CAM corresponds to a larger receptive field/region in the input image. The highest activation (bottom-right) highlights the central-bottom region of the original image.

---

2. Three Custom Sets for CAM

Set 1:
$w^c = [1, 2, 3]$

CAM: $\begin{bmatrix} 4 & 9 \\ 11 & 16 \end{bmatrix}$ → **Normalized:** $\begin{bmatrix} 0 & 0.417 \\ 0.583 & 1 \end{bmatrix}$

Set 2:
$w^c = [3, -2, 1]$
CAM: $\begin{bmatrix} 13 & 1 \\ 7 & 14 \end{bmatrix}$ → **Normalized:** $\begin{bmatrix} 0.923 & 0 \\ 0.462 & 1 \end{bmatrix}$

Set 3:
$w^c = [-1, 3, 2]$
CAM: $\begin{bmatrix} 7 & -1 \\ 14 & 11 \end{bmatrix}$ → **Normalized:** $\begin{bmatrix} 0.533 & 0 \\ 1 & 0.8 \end{bmatrix}$

---

3. Steps of Grad-CAM for the Given Network

Using the original feature maps and computed gradients (example):

- Importance weights (α^c): $\alpha_1 = 1.5$, $\alpha_2 = -0.5$, $\alpha_3 = 2.0$

Grad-CAM before ReLU:

$$\begin{bmatrix} 5.5 & 4.5 \\ 4 & 8 \end{bmatrix}$$

After ReLU: Same (no negative values)

The model focuses strongly on the bottom-right region.

---

4. Grad-CAM when Multiple FC Layers Exist

Grad-CAM works seamlessly even with multiple fully connected layers.
We compute the gradient of the target class score with respect to the last convolutional layer's feature maps. These gradients naturally flow back through all FC layers. Then we apply Global Average Pooling on the gradients to get α_k^c and proceed as usual.

Advantage: Grad-CAM is **architecture agnostic** (works with complex heads), unlike CAM.

---

5. Visualize Before vs After ReLU as Heatmaps

- **Before ReLU**: Contains both positive and negative values → noisy heatmap (red + blue regions).
- **After ReLU**: Only positive values remain → clean heatmap showing only features that **increase** the class score.

Conclusion: ReLU improves interpretability by removing suppressing features.

---

6. What Happens if We DON'T Use ReLU (Full Attribution)

Without ReLU, we get **full attribution**:

- Positive values = features that support the prediction.
- Negative values = features that oppose the prediction.

This gives complete information but produces noisier visualizations. Standard Grad-CAM uses ReLU for cleaner, class-specific explanations.

---

7. Integrated Gradients (IG) Computation

Given: $F(x_1, x_2) = x_1^2 + 2x_2$, $x = (2, 3)$, $x' = (0, 0)$, $m = 10$

Result:

- $IG_1 = \mathbf{4.4}$
- $IG_2 = \mathbf{6.0}$

Total attribution = 10.4 (close to actual output difference 10).

---

8. Three Custom Sets for Integrated Gradients

Set 1: $F = x_1^2 + x_2^2$, $x = (3, 4)$ → $IG_1 \approx 9.0$, $IG_2 \approx 16.0$
Set 2: $F = 3x_1 + 2x_1x_2$, $x = (2, 3)$ → $IG_1 \approx 15.0$, $IG_2 \approx 12.0$
Set 3: $F = \sin(x_1) + x_2^2$, $x = (1.57, 2)$ → $IG_1 \approx 1.0$, $IG_2 \approx 4.0$

---

9. Comparison: Grad-CAM vs Integrated Gradients

Criteria	Grad-CAM	Integrated Gradients (IG)
Purpose	Visual explanation (where?)	Feature attribution (how much?)
Best For	Image data (spatial heatmaps)	Any differentiable model
Computational Cost	Low	Higher (multiple forward passes)
Mathematical Property	Good localization	Axiomatic (complete attribution)
Output	2D Heatmap	Scalar importance per input dimension